We Watch The Bees с серверами Django и Vite

В проекте используются два сервера, каждый из которых отвечает за свою часть приложения.

Vite-сервер (http://localhost:5173/) – это сервер разработки для фронтенда на React. Он запускает клиентскую часть приложения, которая отвечает за интерфейс: отображение ульев, форм, графиков и взаимодействие с пользователем. Vite автоматически пересобирает и обновляет страницу при изменениях кода, что удобно при разработке. Все запросы к API, начинающиеся с /api/, проксируются с порта 5173 на порт 8000, благодаря чему фронтенд может обращаться к бэкенду без проблем с CORS.

Django-сервер (http://127.0.0.1:8000/) – это серверная часть (бэкенд). Он обрабатывает запросы к базе данных, управляет пользователями, хранит информацию об ульях, датчиках и показаниях, а также предоставляет API, через которое фронтенд получает и отправляет данные. Django также содержит административную панель, доступную по адресу /admin. Этот сервер работает на порту 8000 и является основным источником данных.

Оба сервера запускаются через терминал PyCharm через команды:

1. Для backend:
   cd backend
   python manage.py runserver
2. Для frontend:
   cd frontend
   npm run dev

Для тестирования системы был создан «суперпользователь» через терминал PyCharm.

На данный момент **Vite-сервер** позволяет входить в профиль и выходить из него (только суперпользователь), просматривать, добавлять и удалять ульи, а так же редактировать персональные данные. Все разделы кликабельны (включая «Базу Знаний», которая в данной работе не реализовывается), но в основном используются заглушки, кроме тех функций, которые уже работают через API. Ниже представлен вид **Vite-сервера** (рис. 1–11).

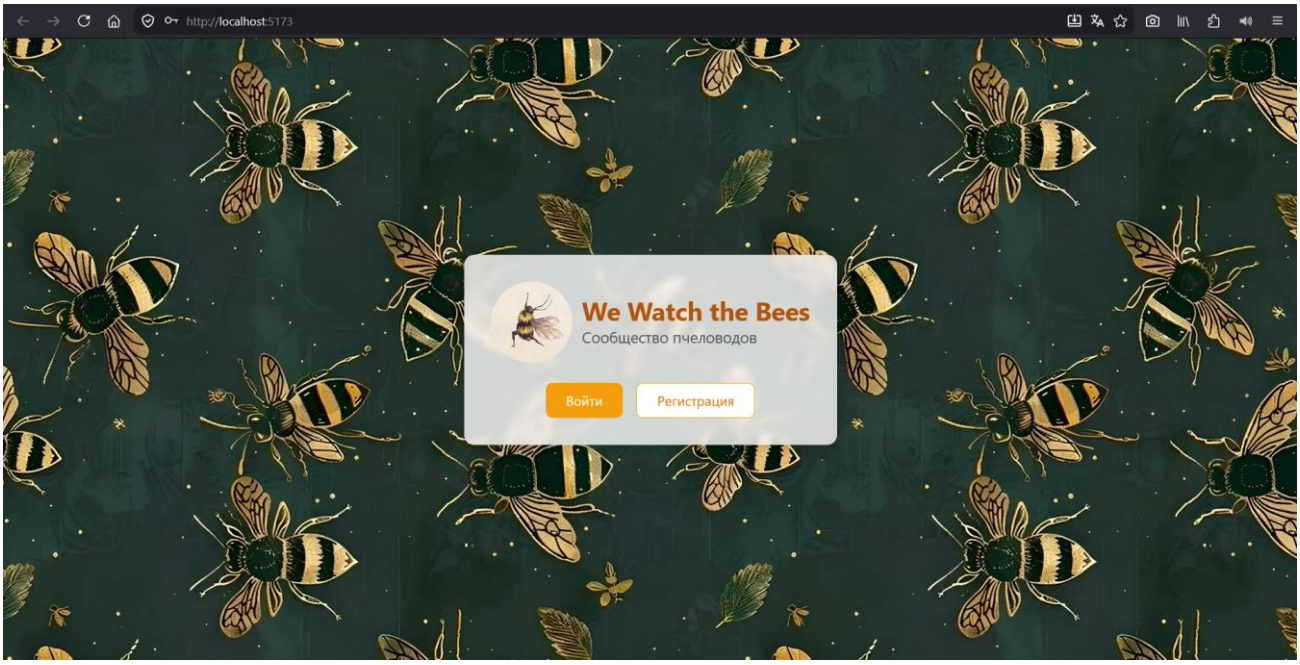


Рис. 1. Начальная страница с кнопками входа и регистрации.

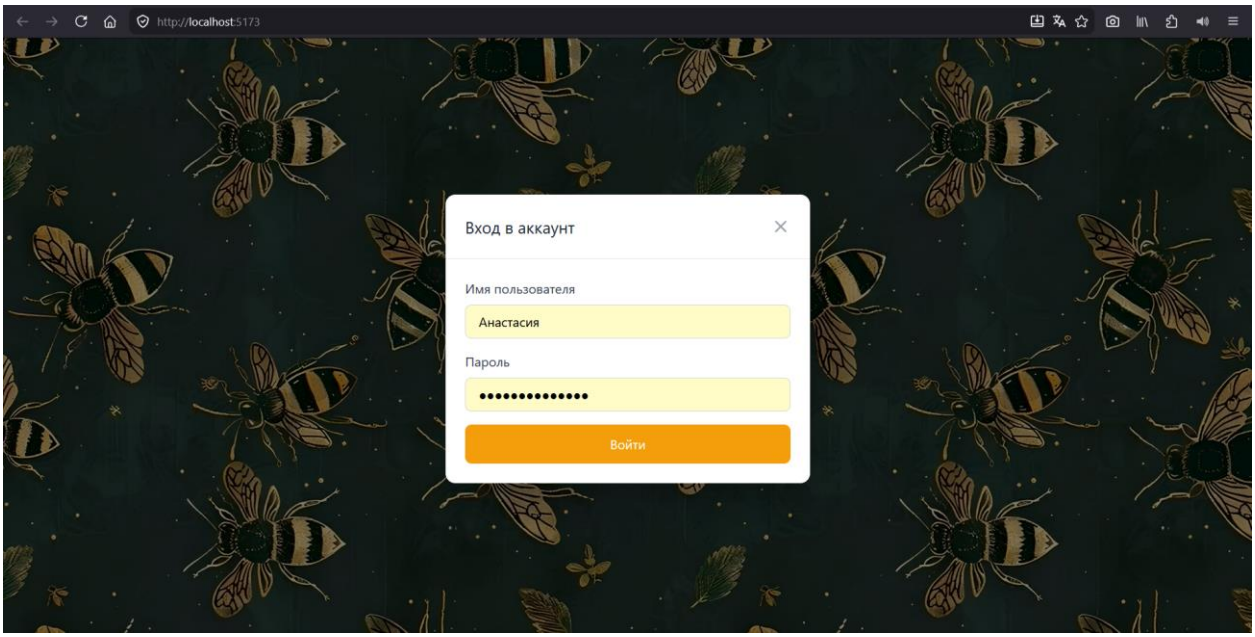


Рис. 2. Начальная страница с формой для входа.

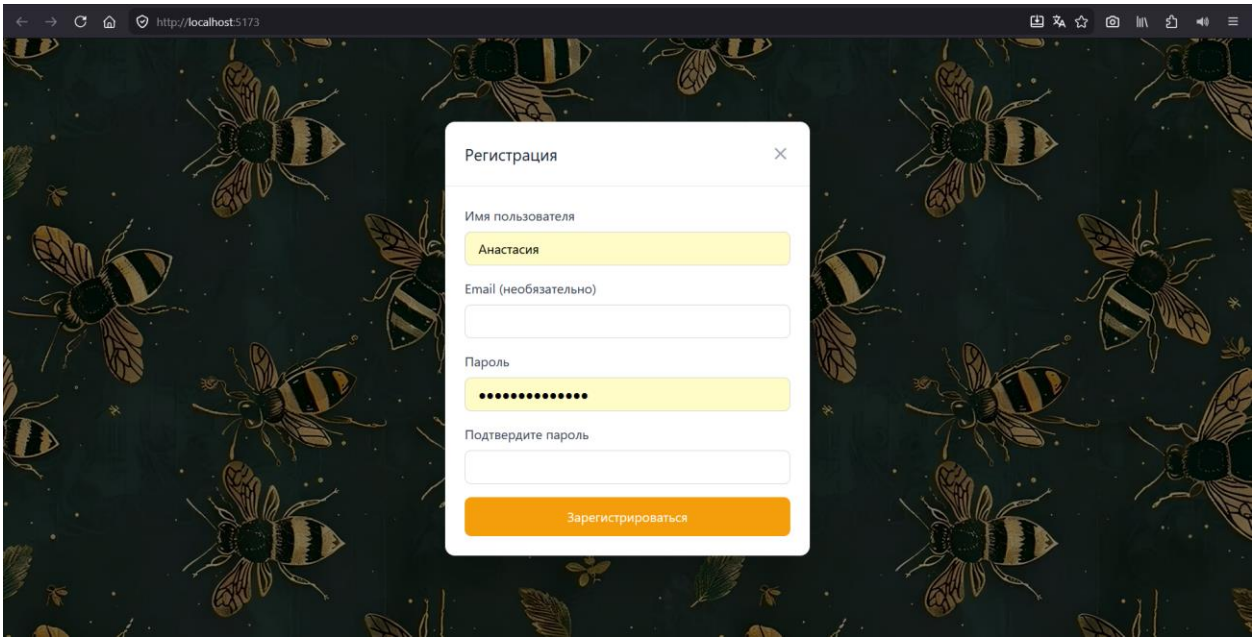


Рис. 3. Начальная страница с формой для регистрации (пока не работает, так как система рассчитана только на тестового «суперпользователя»).

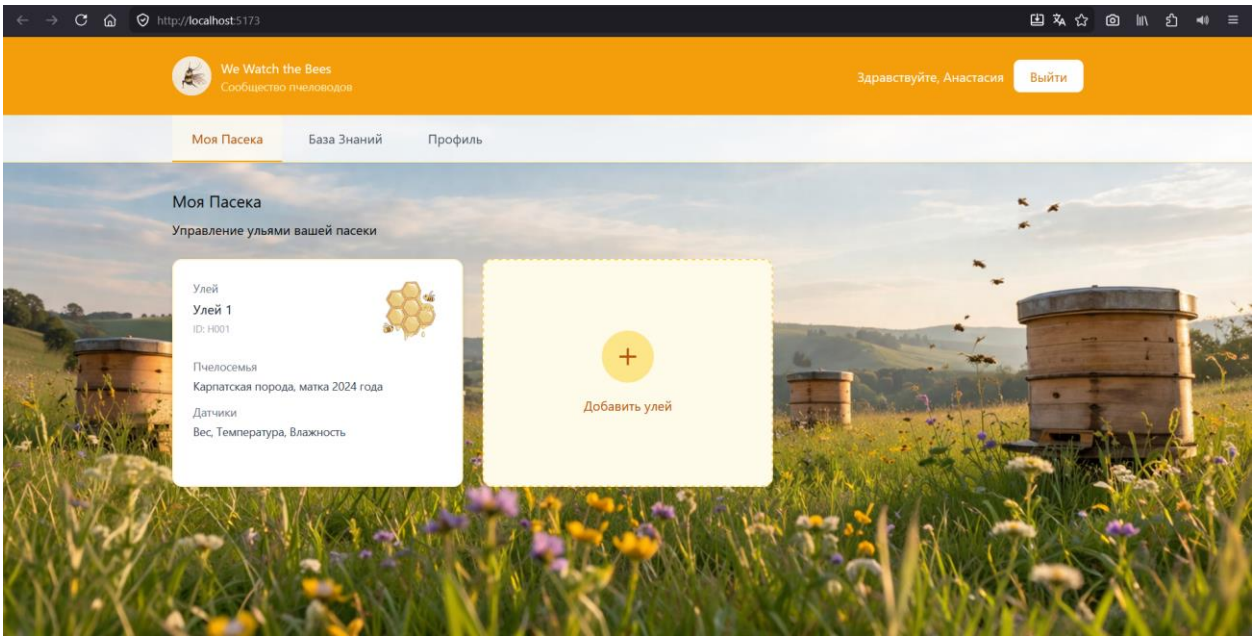


Рис. 4. Главная страница, отражающая дашборд с ульями (по умолчанию главная страница – раздел «Моя Пасека») – при нажатии кнопки выхода пользователя возвращает на начальную страницу.

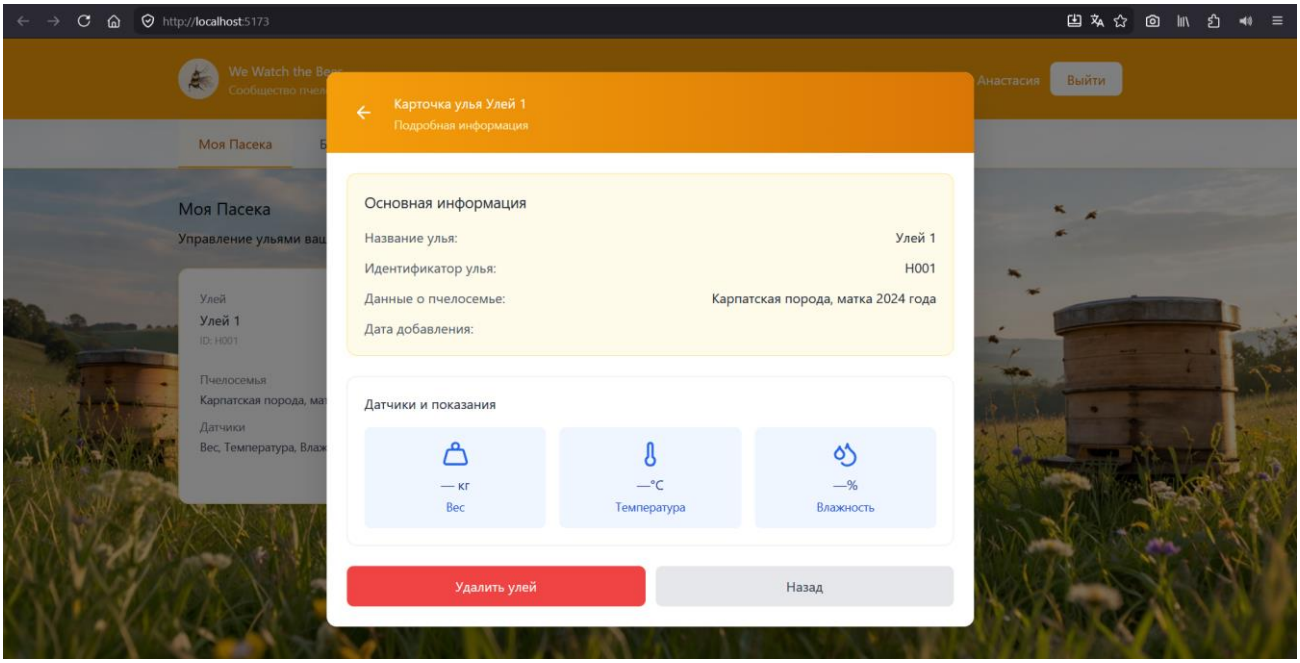
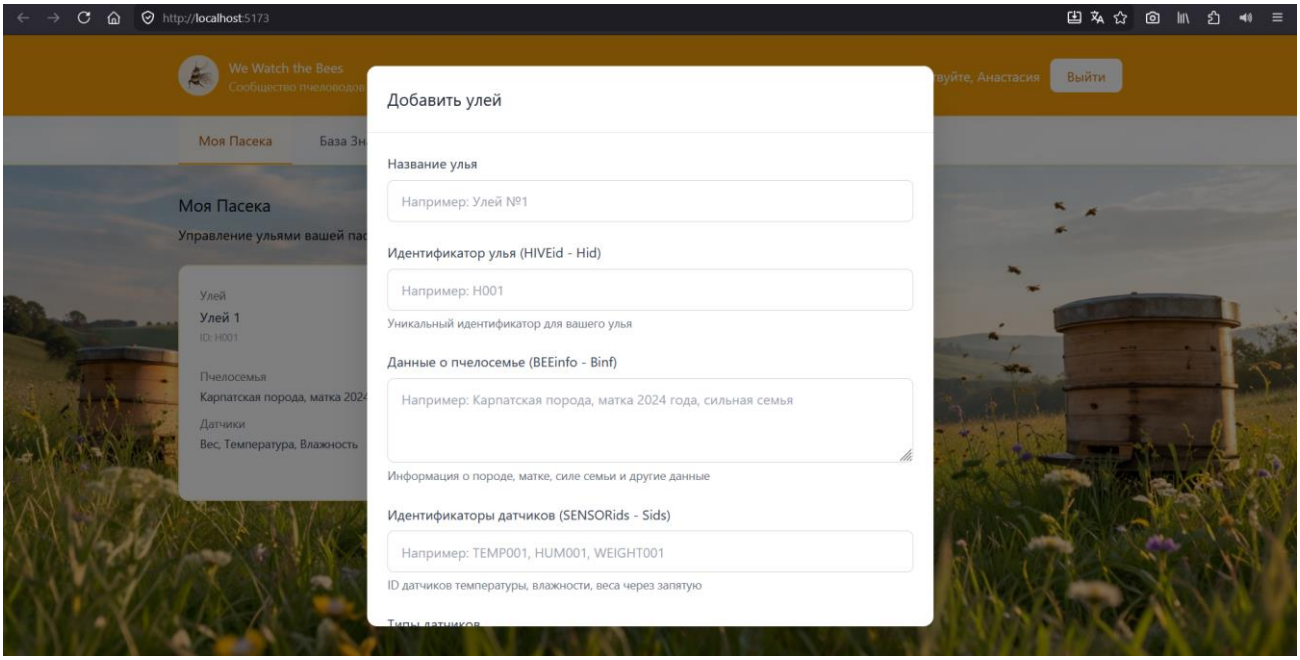


Рис. 5. Просмотр карточки уже созданного улья с возможностью его удаления (впоследствии добавится кнопка редактирования) – датчики ничего не показывают, так как логика обработки показаний будет добавлена позже через эмуляцию.



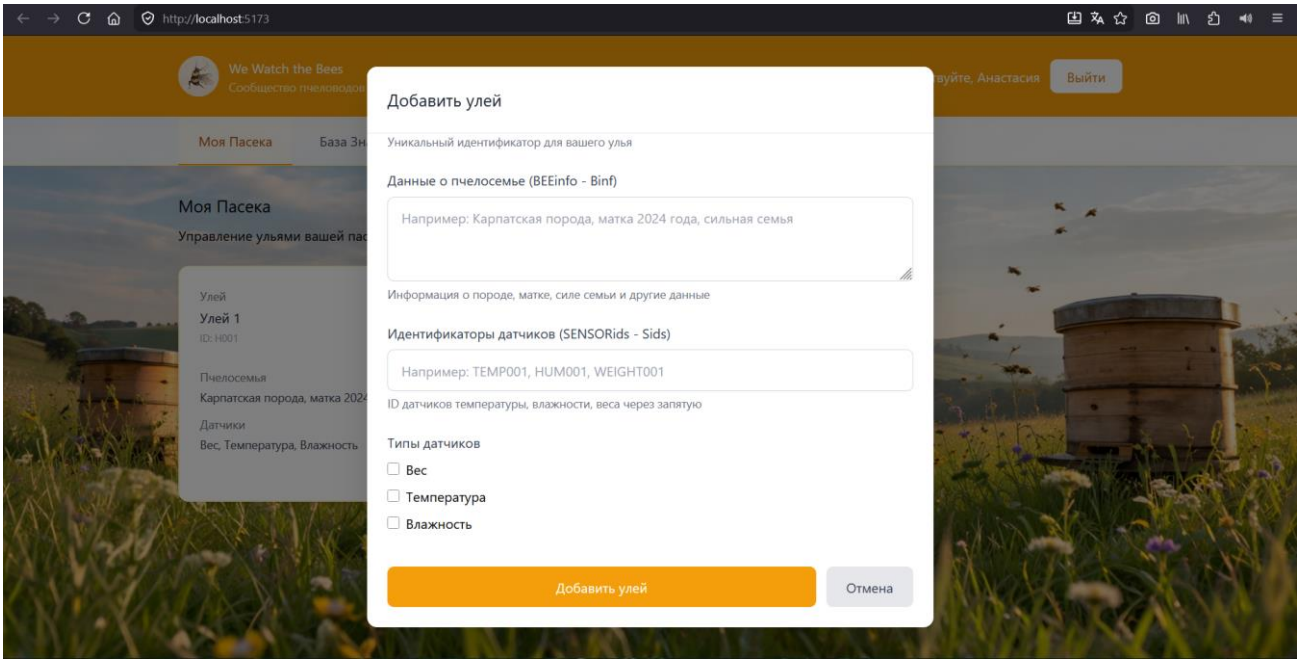


Рис. 6. Форма добавления улья (прокручиваемая) – датчики пока создаются для визуального примера (требуется доработка backend).

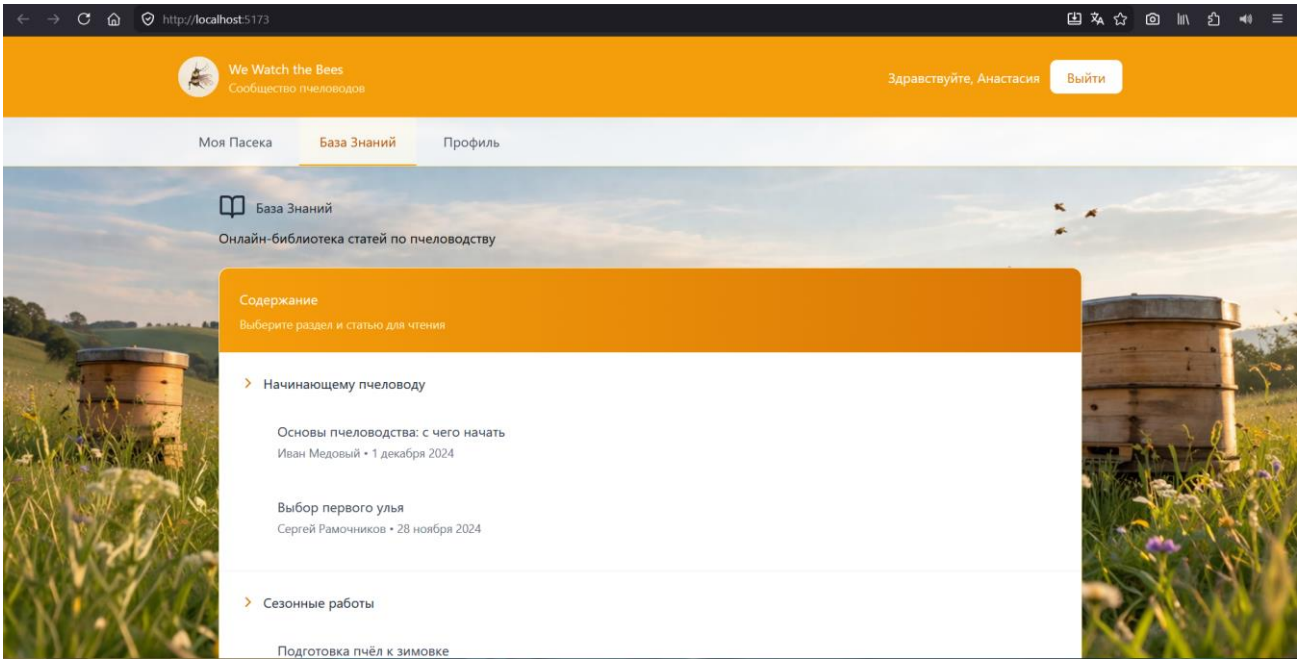


Рис. 7. Просмотр раздела «База Знаний» – он является просто статическим разделом, показанным для примера, так как в данной работе этот раздел реализовываться не будет (фокус сделан на мониторинг пасеки).

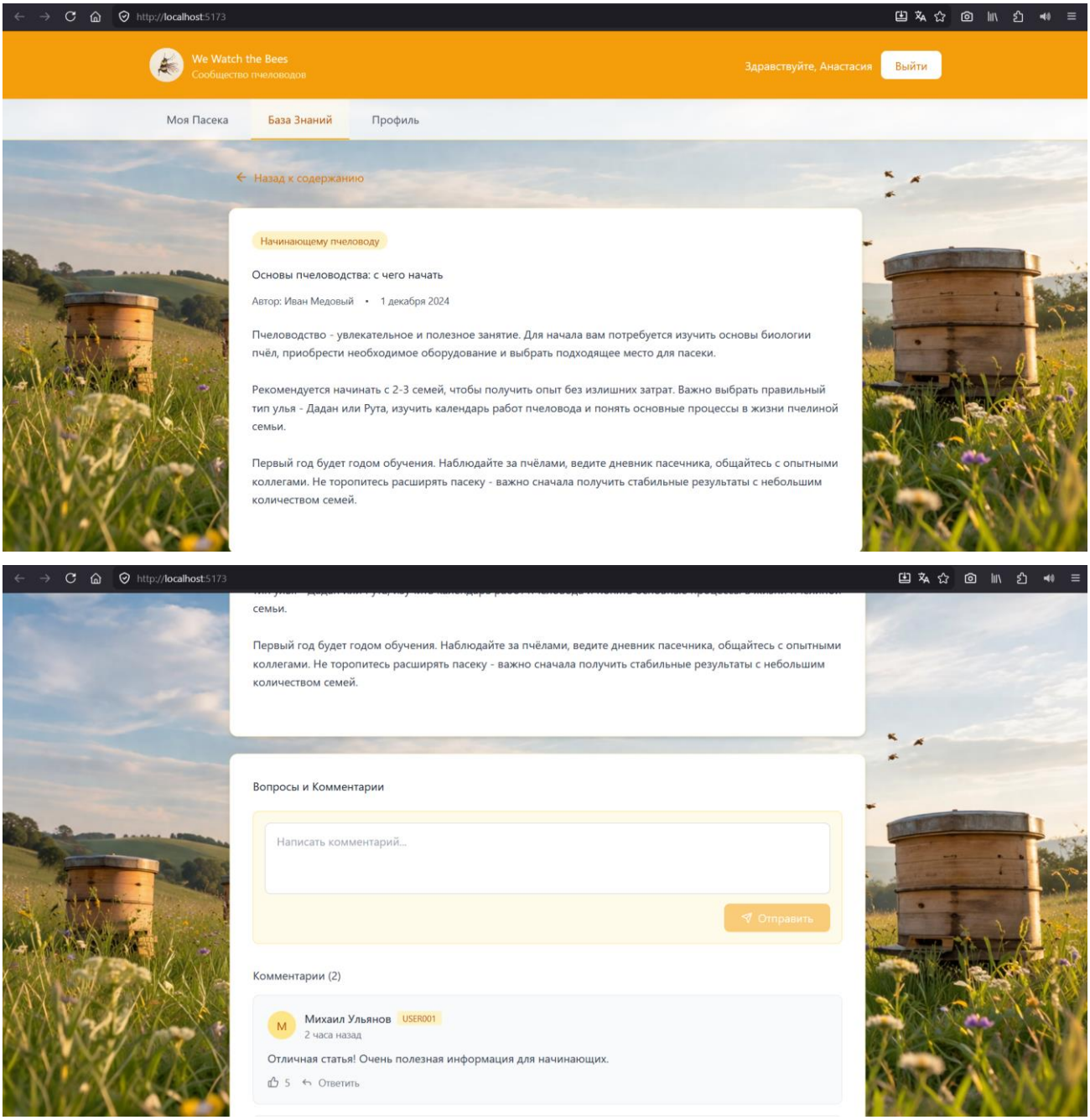


Рис. 8. Просмотр подраздела «Базы Знаний» – статьи и комментариев к ней (тоже статические данные для примера, так как работа фокусируется на мониторинге пасеки).

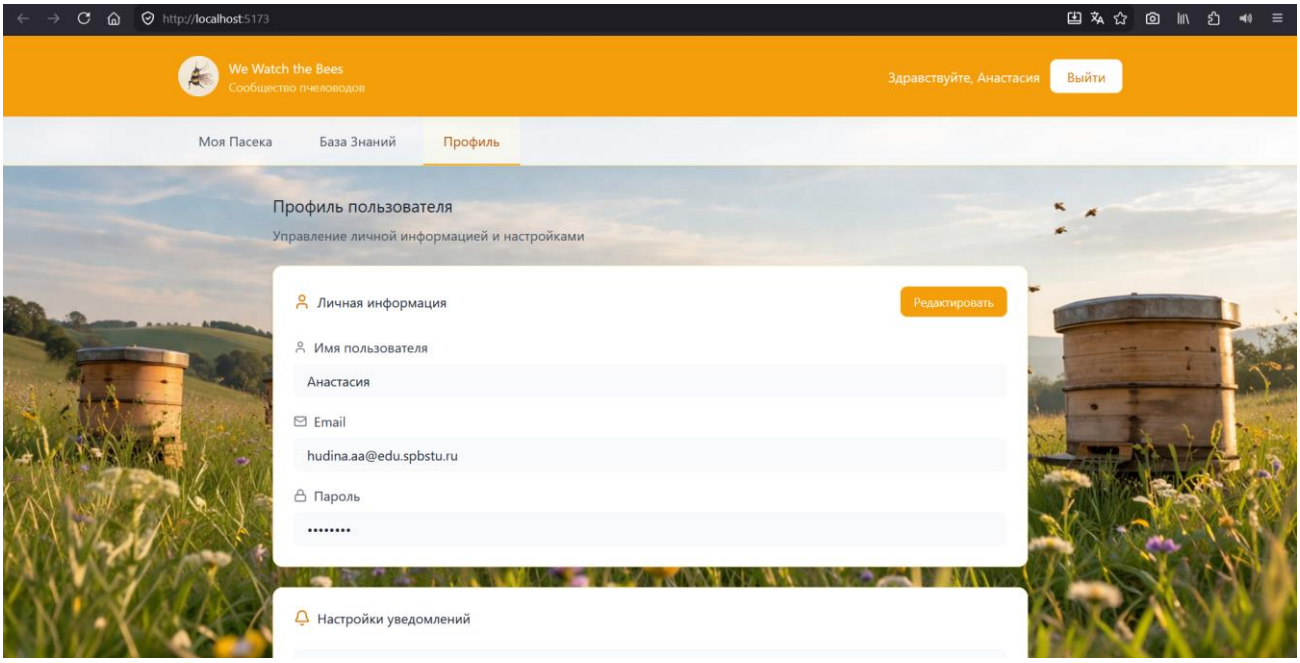


Рис. 9. Просмотр раздела «Профиль» с отображением реальных имени пользователя и email из API и возможностью редактирования данных.

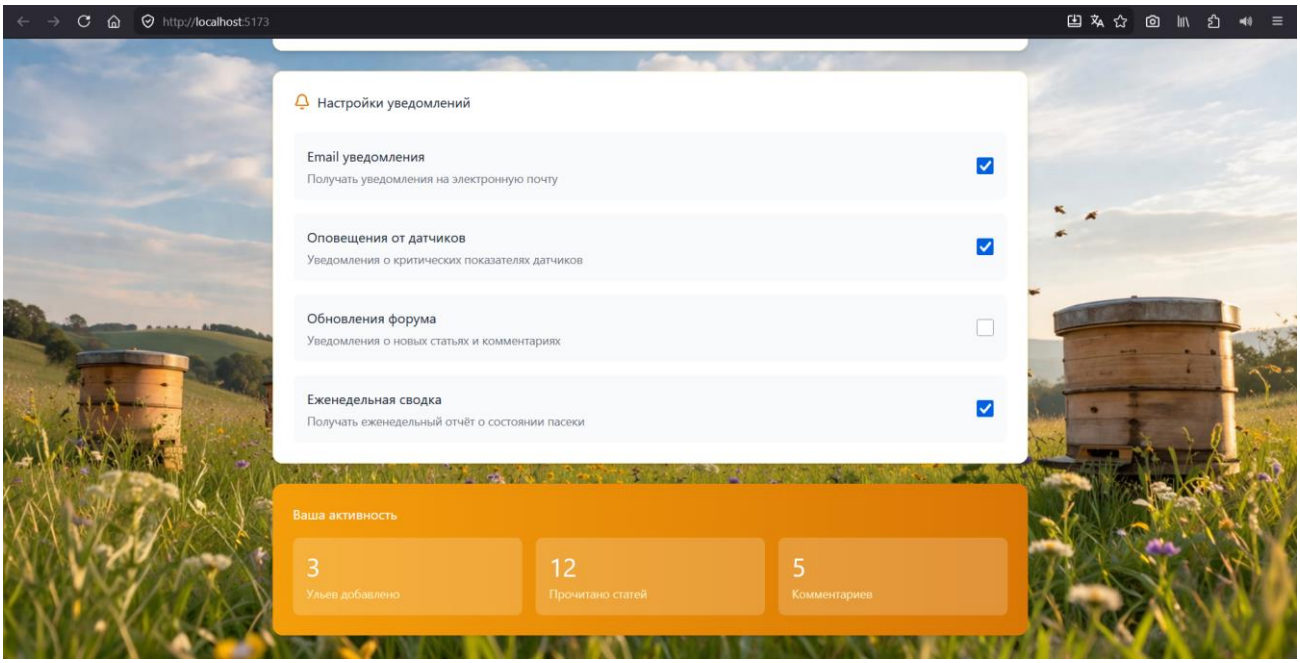


Рис. 10. Просмотр раздела «Профиль» – подраздела с настройкой уведомлений и показом активности пользователя (здесь данные статичны, так как логика настройки уведомлений пока не была реализована, а «активность пользователя» не реализуется в данной работе, как и «База Знаний»).

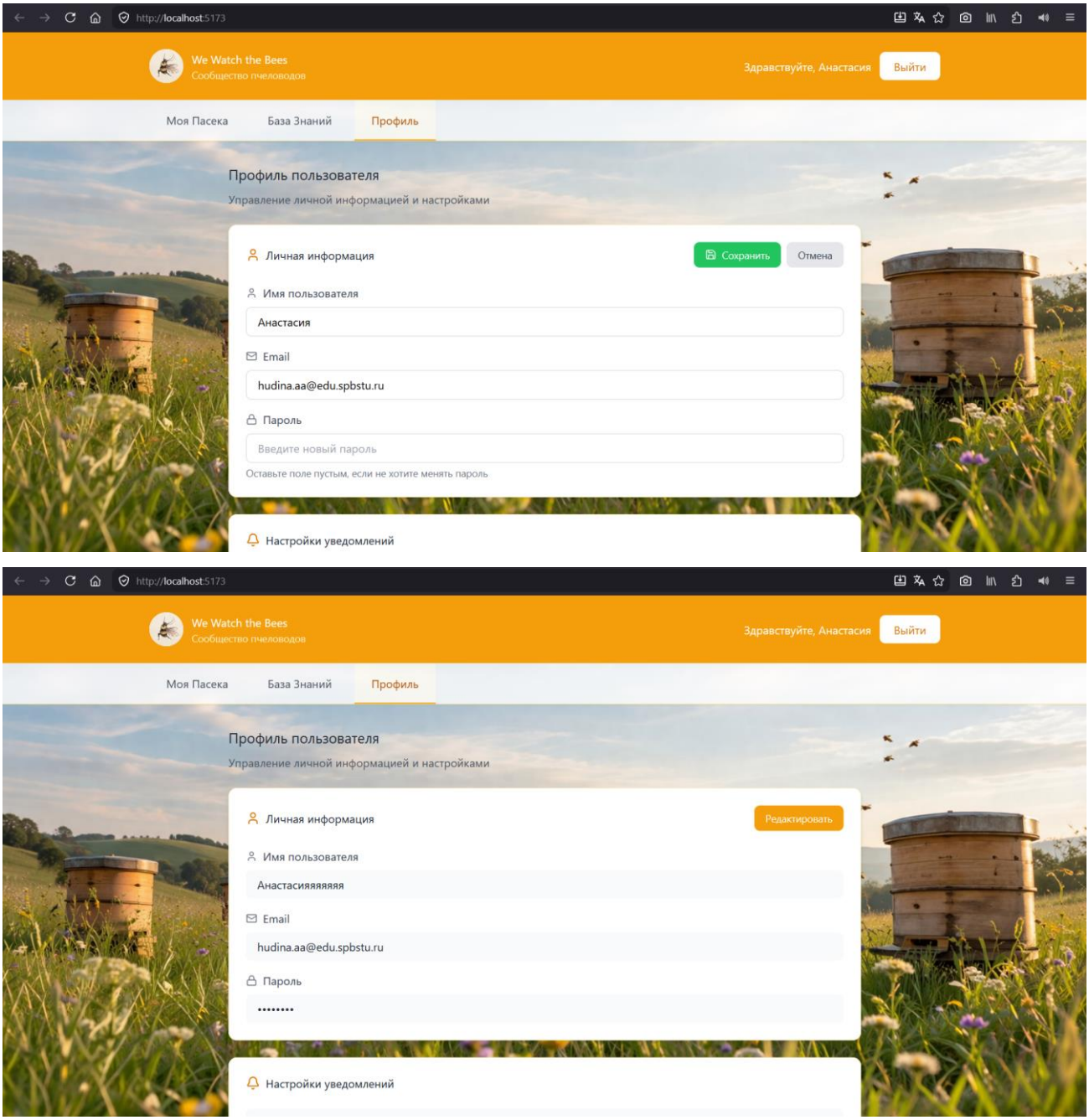


Рис. 11. Форма редактирования персональных данных с их сохранением.

На данный момент **Django-сервер** позволяет войти в профиль, просматривать список ульев, привязанных к пользователю, а так же выходить из профиля. Разделы «База Знаний» и «Профиль» кликабельны, но показывают просто надпись «Раздел будет добавлен позже.», так как пока не были реализованы и подключены. Ниже представлен вид **Django-сервера** (рис. 12–15).

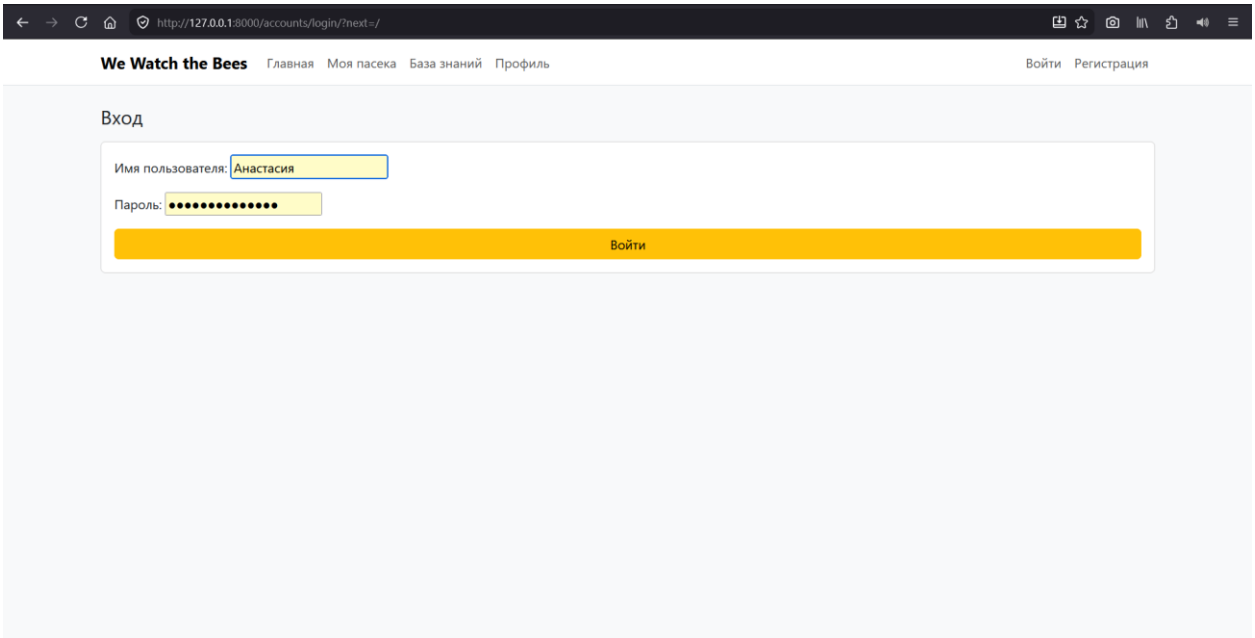


Рис. 12. Вход в профиль суперпользователя.

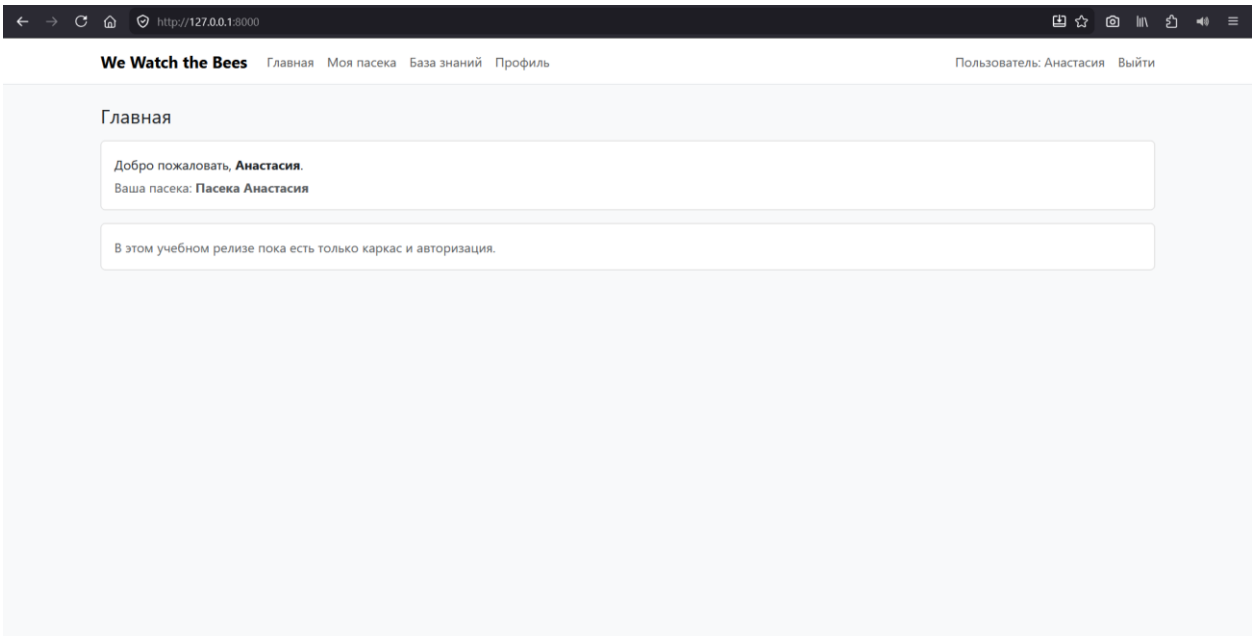


Рис. 13. Главная страница.

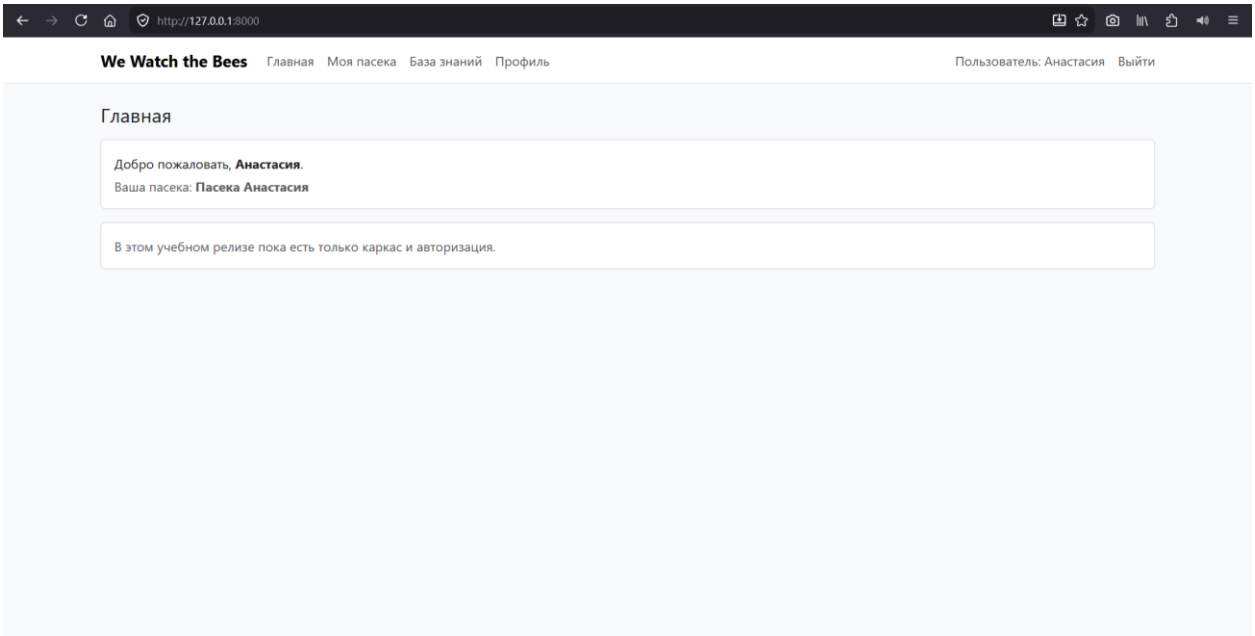


Рис. 14. Страница, отображающая список ульев пользователя.

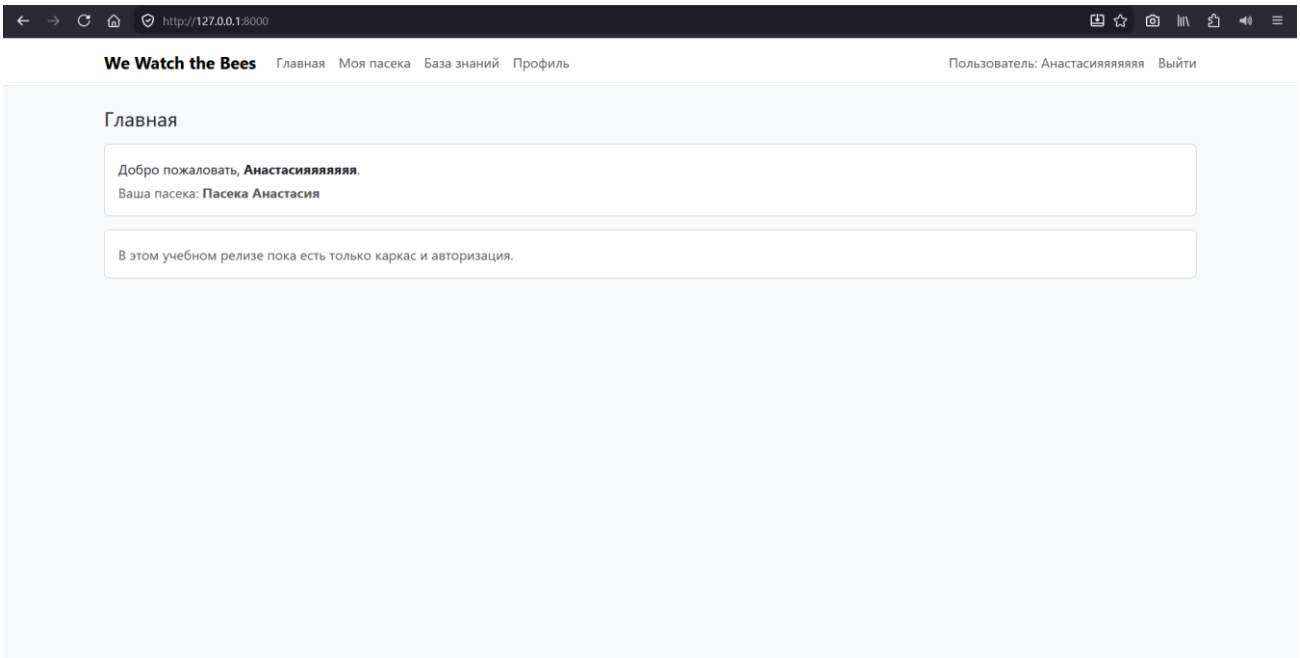


Рис. 15. Дополнение: после изменения имени профиля на сервере Vite – на сервере Django имя пользователя так же было изменено автоматически (аналогично изменяются и добавляются ульи) – что подтверждает реализацию связи backend и frontend через API.

Сводная таблица функциональности

Функция	Статус	Примечание
Регистрация пользователя	Заглушка (UI)	Подтверждение email отсутствует. В системе используется только тестовый суперпользователь.
Вход / выход	Работает (API)	Сессионный вход/выход.
Просмотр профиля	Работает (API)	Имя и email из базы данных.
Редактирование профиля	Работает (API)	Изменение имени, email, пароля сохраняется на сервере.
Список ульев	Работает (API)	Загружается из базы, отображается в карточках.
Добавление улья (название + типы датчиков)	Работает (API)	Создаётся улей и датчики с автоматическими device_id.
Добавление улья (поле hive_id)	Работает (API)	Сохраняется в базе, отображается в карточке.
Добавление улья (поле bee_info)	Работает (API)	Сохраняется в базе, отображается в карточке.
Добавление улья (поле sensor_ids)	Заглушка (UI)	Поле отображается, но не сохраняется; датчики создаются автоматически.
Удаление улья	Работает (API)	Удаляется улей и все его датчики.
Карточка улья – основная информация	Работает (API)	Название, hive_id, bee_info (если есть в БД).
Карточка улья – датчики и показания	Работает (API)	Отображаются типы датчиков и последние показания (если есть). Показания можно добавить через админку; эмуляторы планируются.
Графики показаний	Не реализовано	Будет в следующей итерации.
Оповещения	Не реализовано	Будет с эмуляторами датчиков.

Функция	Статус	Примечание
Эмуляторы датчиков	Не реализовано	Python-скрипты для генерации данных будут реализованы впоследствии.
База знаний	Заглушка (статичные данные)	Фокус на мониторинге, статьи и комментарии не сохраняются.
Настройки уведомлений в профиле	Заглушка (локальное состояние)	Чекбоксы не отправляются на сервер.
Статистика в профиле	Заглушка (статические цифры)	Примеры, не связанные с реальными данными.
Начальная страница / Главная страница + навигация	Реализовано (UI)	Начальная страница для неавторизованных, основной интерфейс после входа.
Адаптивный дизайн	Реализовано (UI)	Корректное отображение на разных экранах.

В текущей версии при добавлении улья **датчики создаются автоматически** на основе выбранных типов (вес, температура, влажность). При этом:

- Каждому датчику присваивается уникальный device_id, генерируемый на стороне сервера.
- В форме добавления улья есть поле «Идентификаторы датчиков (SENSORids)» – это заглушка; введённые пользователем ID датчиков пока не используются и не сохраняются.
- Сами датчики создаются, но показания (SensorReading) не генерируются автоматически. Для демонстрации карточки улья с последними значениями показания нужно добавлять вручную через административную панель Django (в будущем планируется использовать эмуляторы датчиков).
- В карточке улья отображаются все созданные датчики, а если для датчика есть хотя бы одно показание, то показывается последнее значение. Если показаний нет, отображается прочерк.

Таким образом, механизм датчиков реализован полностью (создание, хранение, отображение), но наполнение показаниями в текущей версии не автоматизировано – это следующий этап разработки (эмуляторы датчиков).